



Relatório Detalhado de Atividades

Projeto Flexduino - 25 de Outubro de 2025

Implantação e Teste do Ambiente de Simulação e Desenvolvimento Python

ROBERT NICOLAS DUARTE BASTOS
Nº USP: 15641639

25 de Outubro de 2025

Orientadores:

Prof. Alexandre Alarcon do Passo Suaide
Profa. Kaline R. S. Coutinho

1 Introdução

Este relatório detalha as atividades realizadas em 25 de Outubro de 2025. O foco principal foi a configuração do ambiente de desenvolvimento Python e a tentativa de estabelecer a comunicação entre um script Python local e o firmware do Flexduino a correr no simulador online Wokwi. Este passo é crucial para validar a arquitetura do sistema e permitir o desenvolvimento da interface gráfica ("Software-First").

2 Metodologia e Execução

O trabalho seguiu uma abordagem iterativa: configuração do ambiente, teste de conexão, diagnóstico de erros, implementação de soluções e, face a um bloqueio persistente, a adoção de um plano alternativo (Plano B) para garantir a continuidade do progresso.

2.1 Fase 1: Preparação do Banco de Testes Virtual (Wokwi)

Continuando o trabalho anterior, foi adicionado um potenciômetro virtual ao simulador Wokwi, ligado à porta A0 do Arduino. A execução do firmware simplificado (Versão 4) permitiu controlar a tensão lida (0V-5V) através da manipulação do potenciômetro, fornecendo uma fonte de dados dinâmica e controlável para os testes subsequentes.

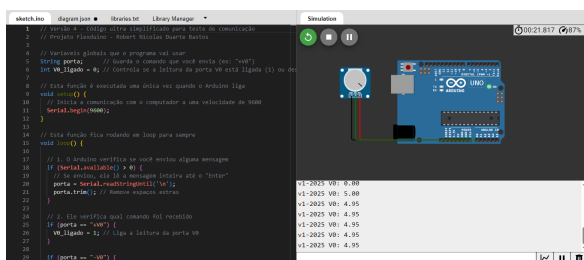


Figura 1: Banco de testes virtual com potenciômetro funcional.

2.2 Fase 2: Configuração do Ambiente Python Local

Foi realizada a instalação do Python 3.14 no sistema operativo Windows. Durante o processo, foram aceites as configurações recomendadas para habilitar caminhos longos e adicionar o Python ao PATH do sistema. Posteriormente, a biblioteca pyserial, essencial para comunicação serial, foi

instalada com sucesso via terminal usando o comando `py -m pip install pyserial`, após uma tentativa falhada com o comando direto `pip`.

2.3 Fase 3: Tentativas de Conexão Python-Wokwi (Plano A)

O objetivo central era conectar o script Python leitor_serial_v1.py à porta serial virtual exposta pelo Wokwi. O Wokwi cria esta porta utilizando um protocolo de rede local (rfc2217), permitindo que programas externos comuniquem com a simulação.

2.3.1 Código de Teste de Conexão

Foi utilizado o script abaixo, que tenta detetar automaticamente a porta ou usar uma porta especificada manualmente.

```
1 # -*- coding: utf-8 -*-
2 import serial
3 import time
4 import serial.tools.list_ports
5
6 # --- Configurações ---
7 porta_serial_desejada = '' # Tenta detetar
8                             # automaticamente
9 velocidade_serial = 9600
10 comando_para_enviar = "+V0"
11
12 # Função para encontrar a porta (adaptada
13 # nas tentativas)
14 def encontrar_porta_serial():
15     # ... (lógica de deteção automática)
16     ...
17     portas = serial.tools.list_ports.
18     comports()
19     # ... (tentativas de identificar Wokwi
20     # /Arduino) ...
21     print("Nenhuma porta encontrada
22     automaticamente.")
23     return None
24
25 # --- Programa Principal ---
26 if __name__ == "__main__":
27     porta_encontrada = None
28     if porta_serial_desejada == '':
29         porta_encontrada =
30         encontrar_porta_serial()
31         # ... (lógica adicional de
32         # verificação Wokwi/físico) ...
33     else:
34         # Tenta usar a porta definida
35         # manualmente
36         porta_encontrada =
37         porta_serial_desejada
38
39     if not porta_encontrada:
40         print("Erro: Porta não encontrada.
41         ")
42         # ... (mensagem de erro e saída)
43         ...
```

```

32     exit()
33
34     arduino = None
35     try:
36         print(f"Tentando conectar à porta
{porta_encontrada}...")
37         arduino = serial.Serial(
porta_encontrada, velocidade_serial,
timeout=1)
38         print(f"Conectado com sucesso!")
39         time.sleep(2)
40         comando_com_newline =
comando_para_enviar + '\n'
41         arduino.write(comando_com_newline.
encode('utf-8'))
42         print(f"Comando enviado: {
comando_para_enviar}")
43         print("Aguardando respostas...")
44         while True:
45             if arduino.in_waiting > 0:
46                 resposta = arduino.
readline().decode('utf-8').strip()
47                 if resposta:
48                     print(f"Arduino: {
resposta}")
49                     time.sleep(0.1)
50         except serial.SerialException as e:
51             print(f"Erro ao conectar/comunicar
: {e}")
52         # ... (restante do tratamento de erros
e fecho da porta) ...
53         finally:
54             # ... (código de fecho) ...
55             input("Pressione Enter para sair
...")

```

Listing 1: leitor_serial_v1.py - Script de teste de conexão.

2.3.2 Processo de Investigação e Despistagem

A detecção automática falhou. Iniciou-se uma investigação sistemática para encontrar e conectar manualmente à porta:

1. **Verificação da Interface Wokwi:** Nenhuma mensagem explícita indicando o endereço da porta (ex: Listening on...) foi encontrada na interface do simulador.
2. **Ferramenta Pyserial (list_ports):** O comando `py -m serial.tools.list_ports -v` foi executado, mas apenas detetou a porta física COM1, não mostrando a porta virtual rfc2217.
3. **Ferramenta de Rede (netstat):** O comando `netstat -ano | findstr "LISTENING" | findstr "127.0.0.1"` revelou duas portas locais à escuta: 7768 e 49780.
4. **Testes Manuais de Conexão:** O script `leitor_serial_v1.py` foi modificado para

tentar conectar explicitamente a cada uma das portas encontradas, usando tanto o endereço IP 127.0.0.1 quanto o alias localhost:

- 'rfc2217://127.0.0.1:49780'
- 'rfc2217://localhost:49780'
- 'rfc2217://127.0.0.1:7768'
- 'rfc2217://localhost:7768'

Todas as tentativas resultaram no mesmo erro: `OSError: (22, 'A sintaxe do nome do ficheiro... é incorreta.', None, 123)`, indicando que o sistema operativo não reconhecia o formato do endereço da porta virtual.

5. **Verificações Adicionais:** Foram ainda verificadas e/ou ajustadas configurações do Firewall do Windows (adicionando regras explícitas para `python.exe` e `py.exe`), desativado temporariamente o antivírus, confirmada a ausência de VPNs/Proxies, atualizada a biblioteca `pyserial` para a versão mais recente e tentada a execução do script Python como Administrador. Nenhuma destas ações resolveu o problema de conexão.

2.3.3 Conclusão da Fase 3

Após esgotar as tentativas de diagnóstico e correção mais comuns, concluiu-se que existe uma barreira de comunicação específica do ambiente local (possivelmente uma configuração de rede, conflito de software ou incompatibilidade entre versões) que impede o `pyserial` de estabelecer a ligação com a porta virtual `rfc2217` exposta pelo Wokwi. Persistir nesta investigação seria moroso e poderia bloquear o progresso no desenvolvimento do software principal.

2.4 Fase 4: Implementação do Plano B (Simulador de Dados)

Face ao bloqueio, decidiu-se adotar uma estratégia alternativa para permitir o desenvolvimento da interface gráfica: criar um script Python (`simulador_dados_v2.py`) que gerasse dados no mesmo formato do Arduino, servindo como uma fonte de dados local para a interface.

2.4.1 Justificativa do Plano B

Esta abordagem permite:

- Manter o progresso no desenvolvimento da interface (GUI) e do gráfico, que são partes complexas do projeto.
- Trabalhar com dados num formato idêntico ao que será recebido do Arduino real.
- Facilitar a transição futura para a leitura da porta serial (física ou virtual), exigindo apenas a alteração da fonte de dados no programa principal.
- Evitar a frustração e a perda de tempo com um problema de conectividade específico do ambiente de simulação.

2.4.2 Código do Simulador

```

1  # -*- coding: utf-8 -*-
2  import time
3  import random
4
5  intervalo_ms = 1000
6  valor_v0_inicial = 2.5
7  variacao_maxima = 0.1
8
9  if __name__ == "__main__":
10     valor_v0_atual = valor_v0_inicial
11     print("Simulador iniciado...")
12     print("(Pressione Ctrl+C para parar)")
13     try:
14         while True:
15             # Simula variacao e limita
16             valor
17             variacao = random.uniform(-
18             variacao_maxima, variacao_maxima)
19             valor_v0_atual += variacao
20             valor_v0_atual = max(0, min(5,
21             valor_v0_atual))
22             # Formata e imprime a linha
23             linha_simulada = f"v1-2025 V0:
24             {valor_v0_atual:.2f}"
25             print(linha_simulada, flush=
26             True)
27             time.sleep(intervalo_ms /
28             1000.0)
29     except KeyboardInterrupt:
30         print("\nSimulador interrompido.")
31     finally:
32         print("Simulador terminado.")

```

Listing 2: simulador_dados_v2.py - Gera dados simulados.

2.4.3 Execução e Validação do Simulador

O script `simulador_dados_v2.py` foi executado com sucesso, imprimindo continuamente no terminal linhas no formato esperado (`v1-2025 V0: X.XX`).

2.5 Fase 5: Desenvolvimento da Interface Gráfica (GUI)

Com o simulador a fornecer dados, foi possível prosseguir com o desenvolvimento da interface gráfica.

2.5.1 Interface V1 e V2: Estrutura e Leitura

Foram criadas versões incrementais da interface (`interface_v1.py` e `interface_v2.py`). A V2 implementou a capacidade de:

- Lançar o `simulador_dados_v2.py` como um subprocesso usando a biblioteca `subprocess`.
- Ler a saída do simulador em tempo real, sem bloquear a interface, usando a biblioteca `threading`.
- Exibir os dados recebidos na caixa de texto da janela Tkinter.

Foram corrigidos erros relacionados com o nome do ficheiro do simulador e com o buffering da saída do subprocesso (resolvido com a flag `-u`).

2.5.2 Interface V3: Adição do Gráfico

As bibliotecas `matplotlib` e `numpy` foram instaladas. O script `interface_v3.py` foi criado, integrando um gráfico `matplotlib` na janela Tkinter. Este gráfico é atualizado em tempo real, plotando os valores de `V0` (extraídos das linhas de dados simulados) em função do tempo.

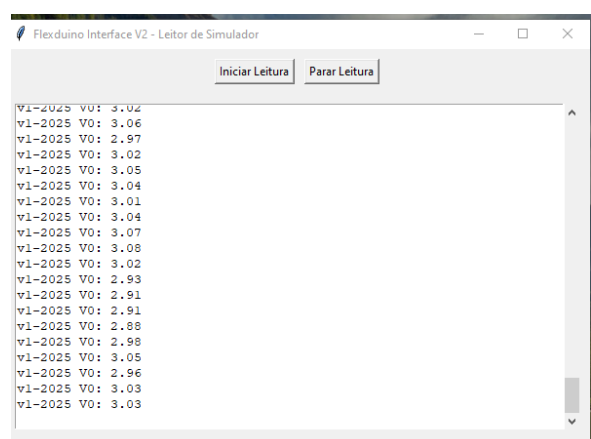


Figura 2: Interface V2 exibindo dados do simulador na caixa de texto.

2.5.3 Código da Interface V3 (Estrutura Principal)

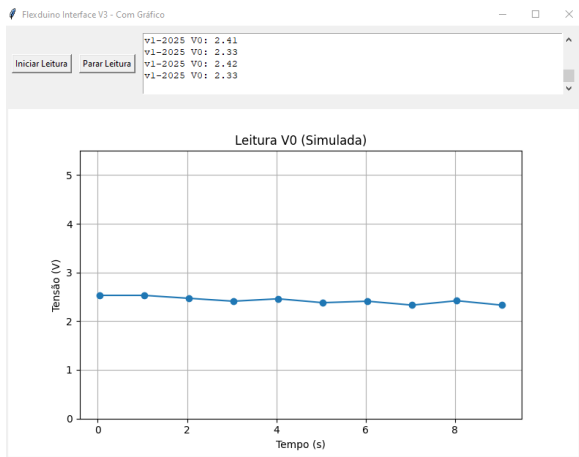


Figura 3: Interface V3 com gráfico dinâmico dos dados simulados.

```

1  # -*- coding: utf-8 -*-
2  import tkinter as tk
3  from tkinter import scrolledtext
4  import subprocess, threading, sys, os,
   time, queue
5  import matplotlib.pyplot as plt
6  from matplotlib.backends.backend_tkagg
   import FigureCanvasTkAgg
7  import numpy as np
8
9  # --- Variáveis Globais ---
10 # ... (processo_simulador, thread_leitura,
   etc.) ...
11 dados_queue = queue.Queue()
12 lista_tempo = []
13 lista_v0 = []
14 # ...
15
16 # --- Funções ---
17 def atualizar_caixa_texto_e_dados(linha):
18     # ... (Atualiza texto e extrai valor
   V0) ...
19     # ... (Adiciona a listas e à queue)
   ...
20     pass # Omitido por brevidade
21
22 def ler_output_simulador():
23     # ... (Loop que lê stdout do simulador)
   ...
24     # ... (Chama janela.after(0,
   atualizar_caixa_texto_e_dados, linha))
   ...
25     pass # Omitido por brevidade
26
27 def atualizar_grafico_realtime():
28     # ... (Lê da queue, limpa e redesenha
   o gráfico ax.plot) ...
29     # ... (Chama janela.after(100,
   atualizar_grafico_realtime)) ...
30     pass # Omitido por brevidade
31
32 def iniciar_leitura_dados():
33     # ... (Verifica se simulador existe,
   inicia subprocess com -u) ...
34     # ... (Inicia thread_leitura) ...

```

```

35     pass # Omitido por brevidade
36
37 def parar_leitura_dados():
38     # ... (Termina subprocesso e junta a
   thread) ...
39     pass # Omitido por brevidade
40
41 # --- Criação da Janela e Widgets ---
42 janela = tk.Tk()
43 # ... (Configuração da janela) ...
44 # ... (Criação de frames, botões, caixa de
   texto) ...
45
46 # --- Configuração do Gráfico Matplotlib ---
47 fig, ax = plt.subplots()
48 canvas = FigureCanvasTkAgg(fig, master=
   janela)
49 canvas_widget = canvas.get_tk_widget()
50 # ... (Posicionamento do canvas_widget na
   janela) ...
51
52 # --- Lógica de Encerramento e Loop
   Principal ---
53 # ... (Função ao_fechar_janela e
   associação com protocol) ...
54 janela.after(100,
   atualizar_grafico_realtime)
55 janela.mainloop()

```

Listing 3: interface_v3.py - GUI com gráfico (Simplificado).

(Nota: O código completo funcional está disponível na pasta do projeto).

3 Resultados e Conclusões

A sessão, apesar dos desafios iniciais de conectividade, resultou num avanço substancial. A adoção do Plano B foi crucial para manter a produtividade. Ao final do dia, foi obtido um protótipo funcional da interface gráfica em Python, capaz de:

- Executar um processo externo (simulador de dados).
- Ler a saída deste processo em tempo real sem bloquear a interface.
- Exibir os dados brutos recebidos.
- Plotar os dados numéricos (tensão V0 simulada) num gráfico dinâmico.

Este resultado representa a conclusão bem-sucedida das metas previstas para as próximas semanas (até meados de Dezembro), colocando o projeto numa trajetória adiantada em relação ao cronograma inicial.

4 Próximos Passos

A próxima sessão de trabalho focará em refinar a estrutura do código da interface e prepará-la para a futura integração com a comunicação serial real.

4.1 Meta para a Próxima Sessão

1. **Refatoração do Código:** Organizar o código da `interface_v3.py` utilizando classes para encapsular a lógica da interface, a gestão do processo/thread e o processamento de dados. Isto tornará o código mais legível, manutenível e fácil de expandir.
2. **Abstração da Fonte de Dados:** Modificar a estrutura para que a interface possa, no futuro, receber dados tanto do `simulador_dados_v2.py` ou tentar ler de uma porta serial real (usando `pyserial`), idealmente através de uma seleção ou configuração.
3. **(Opcional) Melhorias na Interface:** Se houver tempo, explorar pequenas melhorias visuais ou de usabilidade na interface gráfica (layout, feedback ao utilizador).